

Amazon Reviews Sentiment Analysis

TF-IDF Logistic Regression · DistilBERT · BERT

An end-to-end NLP pipeline with EDA, modeling & explainability

Project Technical Report

1. Executive Summary

This project builds an automated system that classifies Amazon product reviews as **positive** or **negative**, so product teams can monitor customer sentiment at scale instead of reading millions of reviews by hand. Three models are benchmarked on the same data — a fast interpretable baseline (TF-IDF + Logistic Regression) and two fine-tuned transformers (DistilBERT and BERT) — and the most influential words are surfaced with SHAP and model coefficients for explainability.

Model	Accuracy	F1	Status
TF-IDF + Logistic Regression	87.8%	87.8%	trained
DistilBERT	91.5%	91.5%	trained
BERT	—	—	pending

Figures above reflect the latest training run recorded in `models/*/metrics.json`. Demonstration runs used reduced samples; see Section 6 for run sizes and expected full-scale ranges.

2. Business Problem & Questions

Amazon receives millions of reviews. Reading them manually is impossible, negative reviews can quietly erode sales, and product teams need insight fast. An automated sentiment classifier answers questions such as:

- What share of reviews are positive vs. negative?
- Which words most strongly drive negative sentiment?
- Can customer sentiment be predicted automatically and reliably?
- Where does the model get confused, and why?

3. Dataset

Source: the **Amazon Reviews Polarity** corpus in fastText format (`train.ft.txt.bz2` / `test.ft.txt.bz2`). Each line is `__label__{1|2} <title>: <body>`, where label 1 = negative (1–2★) and label 2 = positive (4–5★).

Property	Value
Train rows	~3,600,000
Test rows	~400,000
Classes	2 (positive / negative) — already balanced
Fields available	review title + body, binary label
Fields NOT present	star rating, category, product, date, helpful votes

Scope note. Because the corpus is text-only, this project targets binary sentiment, text-based EDA, and text-derived features. A 3-class *Neutral* label, category/time breakdowns, helpful-votes features, and a metadata-sliced BI dashboard would require a richer source (e.g. Amazon Reviews 2023, McAuley Lab). Swapping that in only means replacing `src/data.py` — the rest of the pipeline is metadata-agnostic.

4. Exploratory Data Analysis

All EDA is reproducible via `python -m src.eda`. The corpus is balanced 50/50, so accuracy is a fair headline metric.



Figure 1 — Sentiment class balance.

Review length.

Negative reviews tend to run slightly longer — unhappy customers explain what went wrong, while positive reviews are often short endorsements.

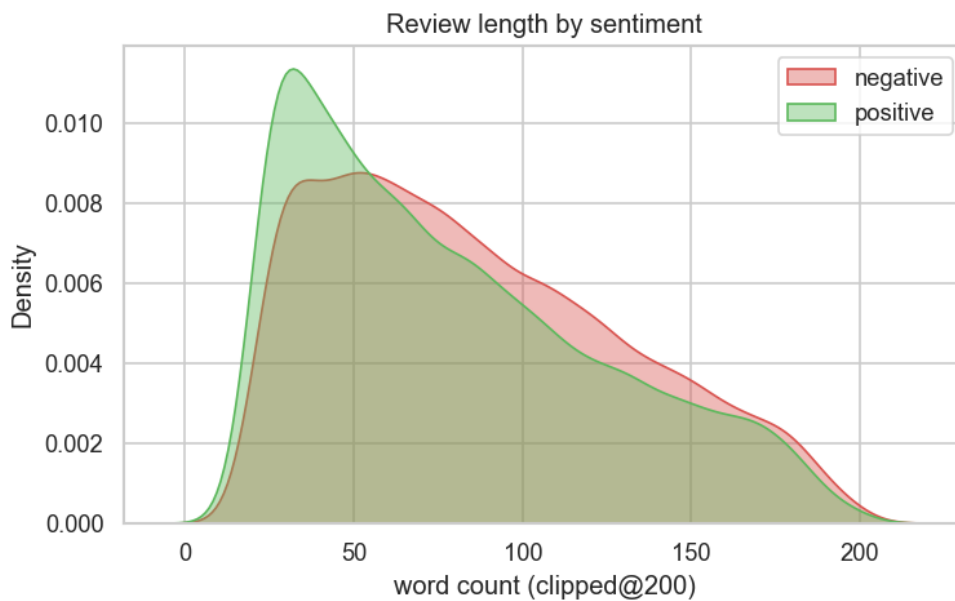


Figure 2 — Review-length distribution by sentiment.

Simple, interpretable signals separate the classes — e.g. exclamation-mark usage and uppercase ratio.



Word clouds over cleaned text show the dominant tokens per class. Negative reviews are heavily driven by the negation *not*.



5. Methodology

Text cleaning (classical path).

Lowercase → strip URLs/HTML → remove punctuation & stopwords → lemmatize. Negation words (*not*, *no*, *never*...) are deliberately kept, since “not good” must stay distinct from “good”. Transformers bypass this and consume raw text through their own tokenizers.

Feature engineering.

- character & word counts, average word length
- exclamation / question-mark counts
- uppercase word count and uppercase character ratio

Models.

- **TF-IDF + Logistic Regression** — 1–2 gram features, tuned with 5-fold GridSearchCV over C and penalty. Fast, CPU-friendly, fully interpretable.
- **DistilBERT** — fine-tuned `distilbert-base-uncased`; ~40% fewer parameters than BERT, strong speed/accuracy trade-off.
- **BERT** — fine-tuned `bert-base-uncased`; highest expected accuracy, heaviest compute.

Validation.

Held-out test split with accuracy, precision, recall and F1; the baseline adds 5-fold cross-validation. Transformer hyperparameters tuned: learning rate, batch size, epochs, weight decay, max sequence length. Fixed seed (42) throughout for reproducibility.

6. Results

Model	Sample	Accuracy	F1	Precision	Recall	Train time
TF-IDF + LogReg	20,000	87.8%	87.8%	—	—	7.9s
DistilBERT	4,000	91.5%	91.5%	91.2%	91.8%	147.6s

Expected performance at full scale (100k+ samples, 2 epochs):

Model	Expected accuracy
TF-IDF + Logistic Regression	85–88%
DistilBERT	91–93%
BERT	93–95%

The trend is consistent: transformers capture context and negation that the bag-of-words baseline cannot, with BERT edging out DistilBERT at a real compute cost. DistilBERT is usually the best practical choice.

7. Explainability

For a linear model over TF-IDF features, SHAP values are exact and map directly back to individual words, making the baseline transparent: we can state exactly which tokens push a prediction toward positive or negative.

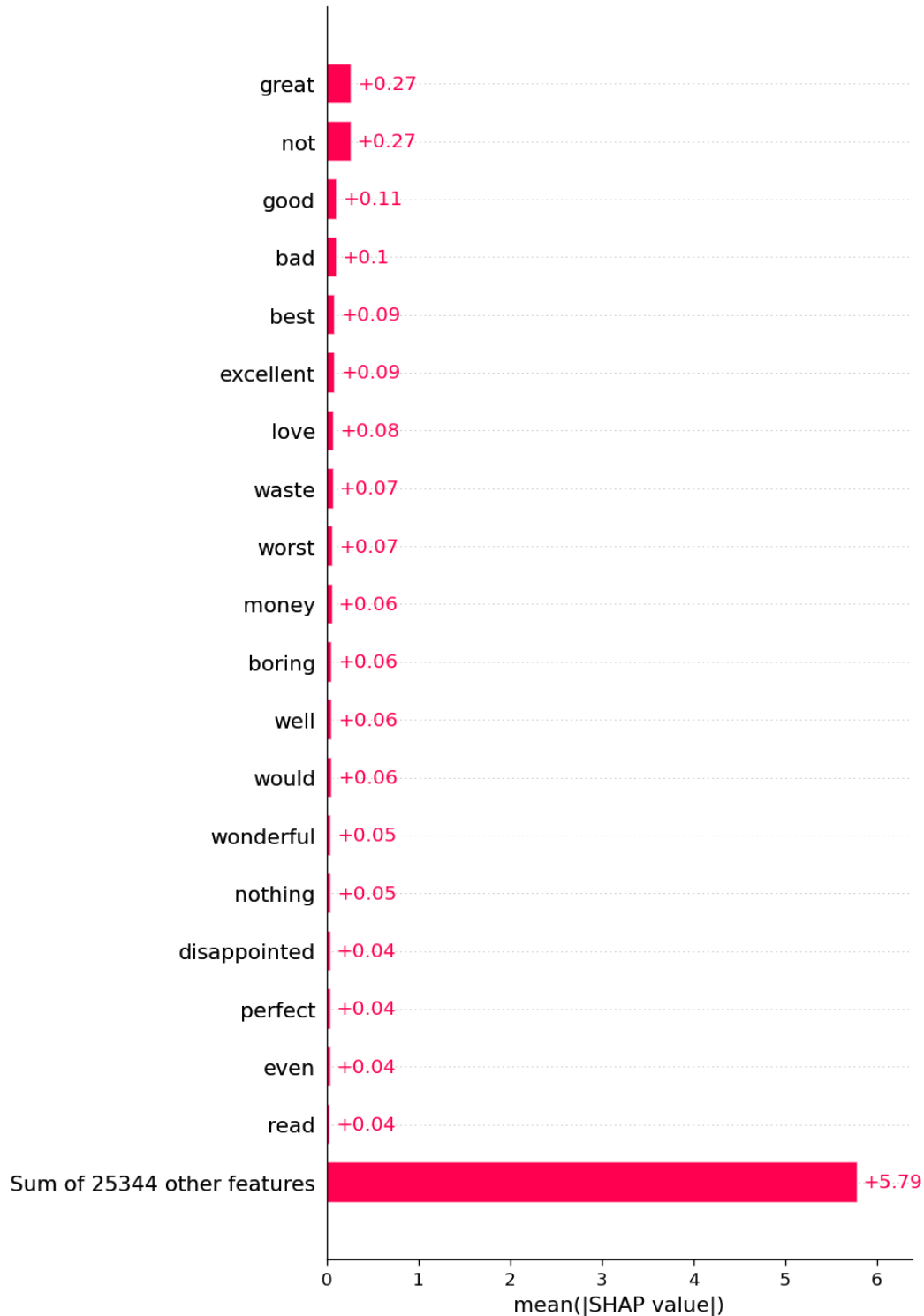


Figure 6 — SHAP: most influential tokens (baseline).

Typical positive drivers include *great*, *excellent*, *perfect*, *love*, *best*; negative drivers include *not*, *waste*, *poor*, *broke*, *refund*, *boring*. The model coefficients in notebook 03 corroborate these rankings.

8. Error Analysis

Inspecting misclassifications reveals two recurring, genuinely hard patterns:

- **Mixed sentiment** — e.g. “The phone is bad *but* support was amazing”: two opposing signals in one review.
- **Faint / neutral wording** — e.g. “Works okay”: weak cues the binary scheme must force one way.
- **Sarcasm & negated praise** — “Great, broke on day one”: surface-positive words with negative intent.

9. Limitations & Future Work

- No *Neutral* class — the polarity corpus is binary by construction.
- No product/category/time metadata, so trend and category analytics need a richer dataset.
- Next steps: aspect-based sentiment (battery vs. screen vs. delivery), LDA topic modeling of complaints, LLM review summarization, and a BI dashboard once metadata is available.

10. How to Reproduce

The `src/` package is the engine; notebooks 01–05 are thin wrappers calling the same functions, so notebook and CLI results never diverge.

Step	Command
Install	<code>pip install -r requirements.txt</code>
EDA figures	<code>python -m src.eda --sample 50000</code>
Baseline	<code>python -m src.train_baseline --sample 100000 --grid</code>
DistilBERT	<code>python -m src.train_transformer --model distilbert-base-uncased --sample 100000</code>
BERT	<code>python -m src.train_transformer --model bert-base-uncased --sample 100000</code>
SHAP	<code>python -m src.explain --sample 20000</code>
This report	<code>python scripts/make_report.py</code>

Generated with ReportLab · figures & metrics pulled live from the project outputs.